EgoQuant: Technical Documentation

System architecture, core algorithms and security design. For the judges of the 2026 Shenzhen FinTechathon, WeBank International Track, Topic B (Wealth Advisory Agent). Status: 20 October 2026. Live system: https://fintechaton-frontend.vercel.app; API: https://web-production-ee5cc.up.railway.app (interactive reference at `/docs`).

This is the overview the organisers asked for in one document. Each section points to the detailed report it summarises: `02_SECURITY_COMPLIANCE_REPORT`, `03_METHODS_AND_MODELS`, `03b_METHODS_QUANT_RISK_ADVISOR` and `09_ON_CHAIN_SEAL` in the same folder. Paths are relative to the backend repository unless stated. The screenshots are from the live site with an invented demo investor.

1. Summary

EgoQuant is a wealth-advisory agent that builds a portfolio a person can understand, question and later prove was given to them unchanged. One rule organises the whole system: **every number a person acts on** (their risk category, their limits, the portfolio weights, the approval and every trade verdict) **is computed by deterministic, tested code, and the language model only explains results it is handed.**

Security decisions were made at the design stage. Each user's data is isolated by the database itself, the AI has no path to change a decision, personal data is replaced by placeholders before any model call, and confirmed advice can be sealed with a fingerprint on a public blockchain so that a later edit is detectable by anyone. The claims below are backed by code and by about 3,540 automatic tests in the backend; the gaps we know about are listed in section 4.10.

2. System architecture

2.1 Components

EgoQuant is split into small components with one job each. Four of them are called agents, but only the advisor and the news tone label use a language model. The others are ordinary programs that give the same output for the same input.

Component	What it does	Uses AI	Code
Risk profile (questionnaire and Risk Game)	Turns answers and game play into a risk category from C1 (most careful) to C5 and a set of numeric limits	No	`egoquant/risk_profile/`
Agent C: market intelligence	Keeps the product catalogue with risk levels R1 to R5, selects the products a person may hold, loads prices and records where every price came from	No	`egoquant/agents/market_intel/`
Agent A: quant	Builds about 100 candidate portfolios inside the limits and simulates each one against the person's goal	No	`egoquant/agents/quant/`, `egoquant/regime/`
Agent B: behavioural twin	Reads the person's real trade history and the crash test, and can only tighten the limits	No	`egoquant/agents/behavioral/`
Risk Engine	Re-checks the finished portfolio independently and has the final say	No	`egoquant/risk_engine.py`
Report	Writes the client report (JSON and HTML) from fixed templates	No	`egoquant/report.py`
Agent D: news briefing	Shows news per holding with a tone label; read-only, never changes a number	Tone label only	`egoquant/briefing/`
Advisor (Ego)	Explains the report in chat, runs what-if scenarios and checks one trade through deterministic tools	Explanation only	`chat.py`, `egoquant/advisor/`
Accounts and seals	Sign-in, saved reports, imported history and consents; fingerprints of confirmed advice on a test blockchain	No	`egoquant/accounts/`, `supabase/`, `egoquant/anchoring/`

2.2 How one portfolio is built

1. The person answers the questionnaire and may play the Risk Game. The profile code produces a category and the numeric limits that go with it.
2. Agent C selects the products the person is allowed to hold (product risk level, trading platform, and the knowledge check for complex products) and loads their price history, recording the source and date of each series.
3. Agent A builds about 100 candidate portfolios, each one solved under every hard limit.
4. Agent B screens the candidates against the person's own behaviour and keeps between 5 and 20 of them. It can lower a limit but never raise one.

5. Agent A simulates every remaining candidate against the person's goal on the same 10,000 simulated futures and picks one by a fixed rule.
6. The Risk Engine recomputes every limit from the chosen weights alone. Only an approved portfolio is shown.
7. The report is written from templates, the advisor receives a signed summary of the report's facts, and the news briefing is built in the background and shown next to the report.

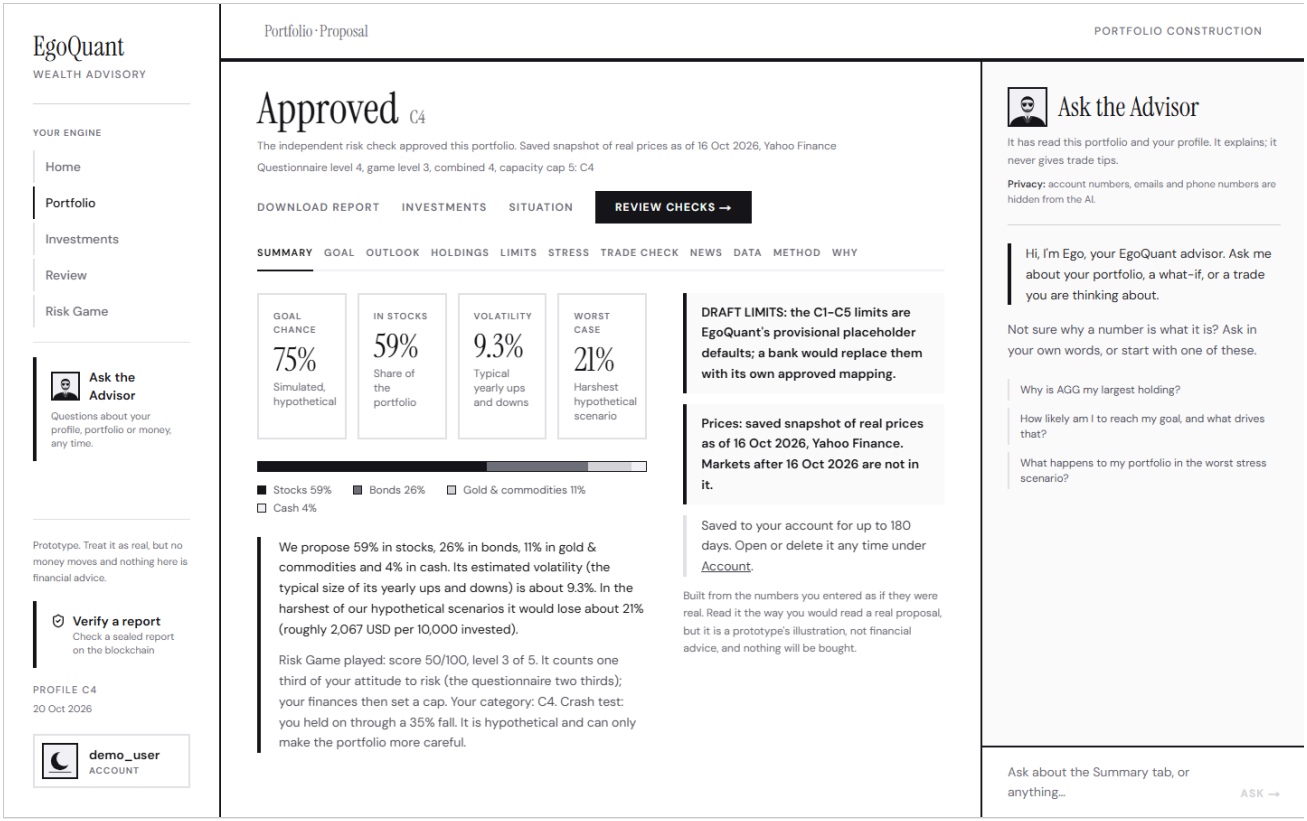


Figure 1. The result of one build: the portfolio approved by the independent Risk Engine, with the report's tabs (Summary, Goal, Outlook, Holdings, Limits, Stress, Trade check, News, Data, Method, Why) and the advisor beside it.

2.3 Deployment and trust boundaries

The system runs on three hosted services: the Next.js frontend on Vercel, the FastAPI backend on Railway and the database and sign-in service on Supabase. The browser never holds a session token. Sessions live in `HttpOnly` cookies set by the frontend's own server routes, which turn them into a Bearer header when they call the backend.

Zone	What it holds	Who can reach it
Browser	The pages and the person's own answers. No token.	The person
Frontend server routes (Vercel)	The session cookies; they check the request's `Origin`, rate-limit sign-in and forward calls to the backend	Deployment owner
Backend (Railway)	All scoring, optimisation, risk checks, trade checks and advisor logic. It stores nothing except a price cache.	Deployment owner
Database (Supabase Postgres and Auth)	Accounts, saved reports, imported trades, consents and logs, with Row-Level Security on every table	Each user, for their own rows only
OpenAI	Engine facts with personal values replaced by placeholders, sent with `store=false`	Provider
Sepolia test blockchain	The label `EGOQ1` and a salted 32-byte fingerprint per sealed report	Anyone, read only

2.4 Technology

- **Backend:** Python 3.14 and FastAPI; NumPy and SciPy for the optimiser and the simulations; `defusedxml` for broker files.
- **Frontend:** Next.js 16, React 19 and Tailwind CSS.
- **Database and sign-in:** Supabase (Postgres with Row-Level Security, and Supabase Auth).
- **AI:** the OpenAI Responses API for the advisor and the news tone label, with FinBERT as the offline fallback for news tone.
- **Prices:** a committed snapshot of real prices for 292 tickers (about 19 years, up to 16 October 2026), with live Tiingo and Binance feeds when allowed.

- **Blockchain:** the Ethereum Sepolia test network.

3. Core algorithms

3.1 Risk category

The questionnaire has 25 questions in four parts. Attitude to risk uses the 13-item Grable and Lytton scale, quoted word for word (score 13 to 47, in five levels). Capacity asks about the horizon, the money that must come back, the emergency fund, the share of total wealth and income stability. Knowledge uses the three Lusardi and Mitchell questions, also verbatim, plus questions on experience. One question asks whether any fall in value could be accepted. Sources and the reason for every cut-off: `egoquant/risk_profile/README.md`.

Attitude and capacity are kept apart, because a person can like risk and still be unable to afford a loss. The category is computed as follows (`docs/RISK_GAME_BLEND.md`, `scoring.py`):

- **Capacity level** (1 to 5) is the average of the capacity answers, with hard caps for money that must be returned and for a short horizon (under one year, or one to three years).
- **Game level** = 1 + game score / 20, rounded down, at most 5. The game score is the average of two scores from 0 to 100: the risk taken, and the behaviour after losing rounds.
- **Attitude level** = (2 × questionnaire level + 1 × game level) / 3, rounded to the nearest whole level. Without a game the questionnaire level counts alone.
- **Category** = the lower of the capacity level and the attitude level. A person who says they could not accept any loss is placed in C1 whatever else they answered.

Because the game carries one third of the weight and the capacity cap always applies, a bold game can raise the category by at most one step and can never lift it above what the person's finances allow.

Questionnaire level	Game level	Capacity	Attitude (blend)	Category
4	2	5	3.33	C3: a cautious game lowers it
3	5	5	3.67	C4: a bold game raises it
5	5	3	5.00	C3: the capacity cap wins
3	not played	5	3.00	C3
any	any	no-loss answer	not used	C1

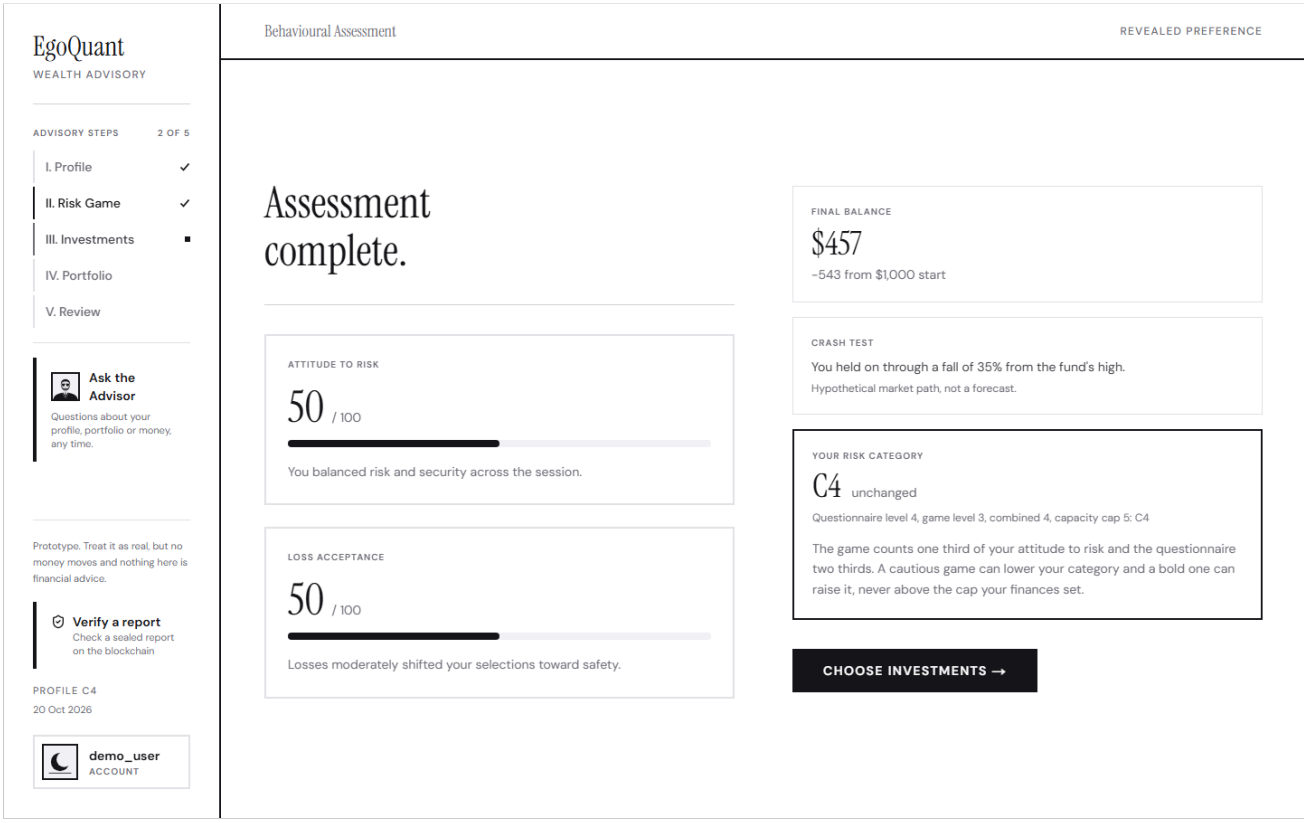


Figure 2. The Risk Game result: the game level is shown next to the questionnaire level, and the category that follows from the rule above.

3.2 Limits for each category

Each category becomes a set of numbers that every later step must respect. The values are our placeholders and are labelled as provisional on every report; a bank would replace them with its own table (`egoquant/risk_profile/config.yaml` ; a test keeps Agent A's copy equal).

Category	Highest product level	Stocks (max)	Gold and commodities (max)	Crypto (max)	Volatility (max)	Loss in the crash scenario (max)
C1	R1	0%	0%	0%	4%	5%
C2	R2	0%	0%	0%	7%	10%
C3	R3	40%	10%	0%	10%	20%
C4	R4	70%	15%	0%	13%	35%
C5	R5	90%	20%	5%	17%	45%

The crash scenario is a hypothetical shock in which stocks fall 30%, bonds 5%, gold and commodities 15% and crypto 60%. Four more hypothetical scenarios are always shown. No single holding may exceed 40% (C1, whose only products are two short-term Treasury funds, may hold one fund at 100%), each single stock is capped at 10%, and complex products such as crypto require passing the knowledge check.

3.3 Universe and prices (Agent C)

Agent C keeps a catalogue of funds, single stocks and crypto-assets, each with a product risk level from R1 to R5. For a given person it keeps only the products whose level the category allows, that trade on the platforms the person chose and, for complex products, that the knowledge check permits. It then walks the person's interest areas, most traded products first, until the universe is full (at most 100 instruments).

On the live server prices come from a committed snapshot of real historical prices, so a build takes seconds and cannot be slowed down by a third party. When live feeds are allowed, a time budget applies: after 15 seconds of waiting, or two slow requests, the whole build switches to the snapshot and says so. Sources are never mixed within one build, and every report names the source and date of its prices.

3.4 Candidate portfolios (Agent A)

Agent A is a numerical optimiser. It estimates the covariance of returns from about 20 years of daily history, with shrinkage, and treats expected returns as an explicit assumption, never as the historical average. It then builds a menu: 20 points on the risk axis (the share in stocks, from zero up to the category's cap) times 5 construction styles. Each candidate is a long-only portfolio solved under every hard limit with SciPy's SLSQP solver after a linear-programming feasibility step. The styles are lowest risk, diversified (with a penalty on concentration), equal weight, inverse volatility, and a tilt toward the person's interest areas. After duplicates are removed about 100 candidates remain (`egoquant/agents/quant/agent.py`).

When the recent volatility of the market is high compared with its long-run level, the volatility cap is reduced for that build. It is never increased.

3.5 Behavioural tightening (Agent B)

People answer questionnaires one way and trade another way. Agent B reads what the person actually did, from an Interactive Brokers trade report that they upload or fetch once with a read-only token, and how they played the crash test in the app. It turns that evidence into extra limits with fixed rules; no AI is involved.

Behaviour	Rule	Limit it creates	Strength
Panic selling	Sold at least 30% of their stocks within 10 trading days while the account was already down at least 5%. Selling to fund a withdrawal does not count.	Maximum loss in the crash scenario = the median loss at which they sold	Hard after two or more episodes (confidence 0.8, or 0.9 for three or more); soft after one (0.6)
Crash test in the app	Sold at least 30% of the fund in the hypothetical crash. Used only when there is no informative trading history.	The same kind of loss limit, never more than one category below the questionnaire's	Soft (0.55)
Overtrading	One-way turnover above 100% a year	At most 20% of the portfolio traded per rebalance	Soft (0.6)

Tests enforce three properties (`tests/agent_b/` , 680 tests). A behavioural limit is only sent when it is stricter than the category's own limit, so evidence can never loosen anything. A hard limit is never relaxed, while a soft one may be relaxed if no portfolio exists, and the report says so. And Agent B cannot make the problem impossible: if even the safest candidate loses more than the evidence allows, the limit is set to that candidate's loss and flagged. Agent B then keeps between 5 and 20 candidates whose loss in a severe fall sits inside the person's comfort band.

3.6 Goal simulation and the final choice

To estimate how likely each candidate is to reach the person's goal, EgoQuant replays the past. It builds 10,000 simulated futures from month-long blocks (21 trading days) of real daily returns, which keeps crashes and rebounds together the way they happened. Every candidate runs on the same futures with a fixed seed, so differences between candidates come from the portfolios and not from luck. Each month the portfolio earns its block's return, is rebalanced to its target weights, pays its fund fees and receives the monthly contribution or withdrawal (`egoquant/agents/quant/goal.py`).

Four adjustments keep the use of history honest:

- **Regime-aware sampling.** A four-state hidden Markov model (calm, normal, stressed, crisis), fitted offline and loaded from a hash-checked file, decides which kind of month comes next, so that crises last as long as they did in the past. If anything is wrong with the model file, the simulation falls back to the plain block bootstrap and the report says why (`egoquant/regime/`).
- **Rate anchor.** Bond and cash funds keep their real daily moves but take today's market interest rate (plus a term premium for bonds) as their average, because their past averages reflect past interest rates.
- **Hindsight correction.** Stocks, equity funds and crypto are picked from today's most-traded list, which is a list of past winners. They keep every real daily move but take the average return their sensitivity to the world stock market earned (zero alpha).
- **Short histories.** A young fund does not shorten everyone's history. Its missing years are rebuilt by regression on long-history funds, with resampled residuals and zero alpha.

The chosen portfolio is the candidate with the highest share of futures that reach the goal. Among candidates within one percentage point of the best, the one with the smaller shortfall in the worst 5% of futures wins, then the one with the lower volatility. The result is shown as a natural frequency ("N in 100 simulated futures") together with what would change it: the monthly amount needed for a 50, 75 or 90 in 100 chance, the effect of a later goal date, and the difference between results on the older and the newer half of history. It is always labelled as a simulation on past data, never as a forecast.

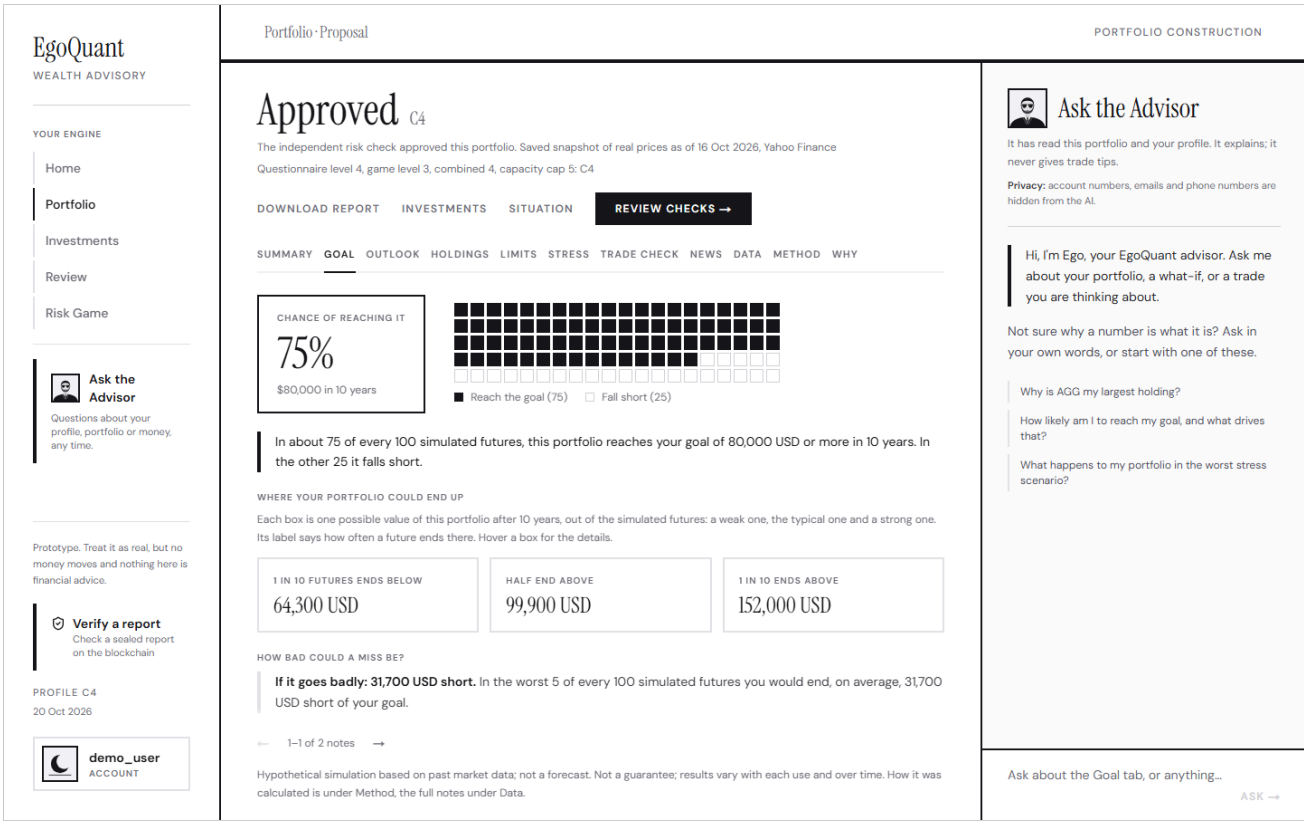


Figure 3. The Goal tab: the chance of reaching the goal as a natural frequency (75 in 100 simulated futures), the range of end values and the average size of a miss in the worst 5 futures in 100.

3.7 Independent Risk Engine

The Risk Engine is a separate program (`egoquant/risk_engine.py`). It does not call Agent A, Agent B or their maths; it takes the proposal's weights, the products' metadata and the limits, and checks the following:

1. The weights are long-only and sum to 1.
2. Every holding's product risk level is within the person's maximum. An unknown product or a missing level counts as a violation, so the check fails closed.
3. No holding is above its own cap.

4. The total in stocks is within every hard limit.
5. Gold and commodities, crypto and cash are within their class caps.
6. The loss in every named stress scenario is within every hard limit.

If any check fails the portfolio is not shown, and the person reads the reason in plain words instead of an error code.

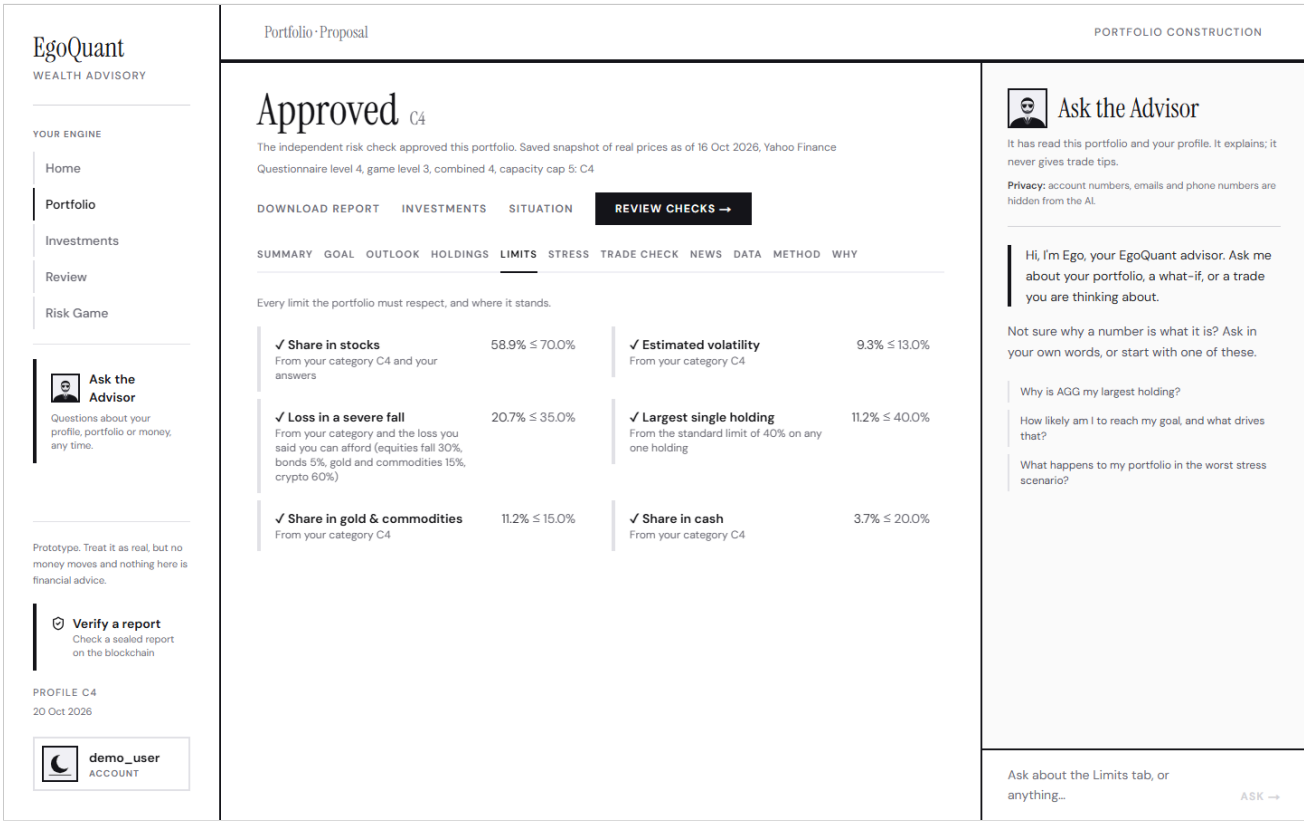


Figure 4. The Limits tab: every limit the portfolio must respect, where it comes from, and where the portfolio stands against it.

3.8 Trade check

A person can ask whether a planned trade fits their limits. Their holdings come from their saved trading history, never from the request. The planned trade is applied, and the portfolio after the trade is judged against the same limits and reviewed again by the Risk Engine. The answer is one of three verdicts: *within your limits*; *within your limits, with warnings* (for example, a limit that is still exceeded but which this trade reduces, or little room left); or *outside your limits* (a hard limit broken, or an existing breach made worse). Every reason carries its numbers: the limit, the value before and the value after. The check never tells the person to trade or not to trade, and EgoQuant has no way to place an order. It was property-tested on 5,000 random trades (`tests/system/test_trade_check_sweep.py`).

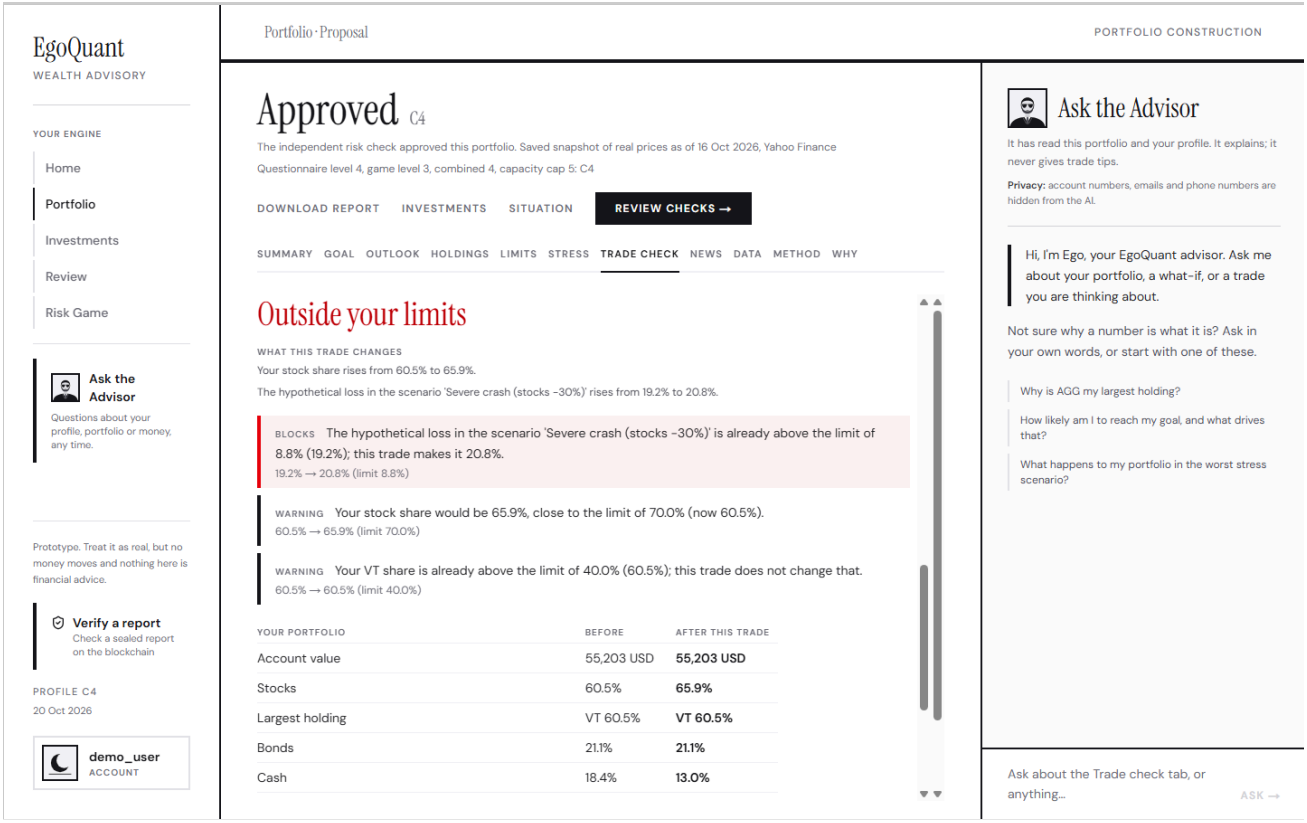


Figure 5. The trade check: the portfolio before and after one planned trade, measured against the same limits, with a verdict and its reasons.

3.9 Advisor

The advisor, Ego, answers questions about the report in plain language. It receives the engine's facts in a package signed by the server with an HMAC, so the browser cannot alter them. It has two tools. `run_what_if` re-runs the deterministic engine on a validated scenario (an extra monthly amount, a one-off amount, a different horizon or goal, or excluded products). `check_trade` runs the trade check above for signed-in investors. Neither tool has a field for a limit, a category or an identity, and at most two tool calls are allowed per message. The safeguards around the model are described in section 4.5.

3.10 News briefing

Agent D collects headlines for each holding and for the person's interest areas from sources rated by reliability, and labels each headline as positive, negative or neutral for holders of one named fund. This label is the only place where a language model's output becomes a number: it uses a fixed prompt pinned by its SHA-256 hash, temperature 0 and a shared cache, with FinBERT as the automatic fallback. On 120 hand-labelled headlines it agreed with the team's labels 85% of the time, against 53% for FinBERT. The label feeds only the news tab's tone summary and an alert that offers a risk-profile re-check. It never changes a category, a limit, a weight or the Risk Engine's verdict (`docs/AGENT_D_LIMITATIONS.md`).

4. Security design

EgoQuant handles personal financial information and gives advice that a person may act on, so most design decisions were made with security and compliance in mind. This section sets out the principles, then each area of control with the evidence behind it, and ends with the gaps we know about. The full report with an evidence tag on every claim is `02_SECURITY_COMPLIANCE_REPORT` .

4.1 Principles

1. **Decisions are code, explanations are AI.** Scores, categories, limits, weights, approvals and trade verdicts come from deterministic, tested code. The language model only explains results it is given.
2. **Suitability comes first.** The category is the lower of what a person's finances allow and how they feel about risk, and "I could not accept any loss" always gives the most careful category.
3. **An independent check has the final word.** The Risk Engine recomputes every limit from the weights alone, and only an approved portfolio is shown.
4. **Behaviour can only tighten.** A trade history, the crash test and the game can lower a limit, never raise it.

5. **One person can never read another's data.** Every database call runs as the signed-in user under Row-Level Security, the backend filters again, and no all-powerful database key exists in the system.
6. **Less data, kept for less time.** Broker passwords are never asked for, tokens are used once, uploads are parsed in memory and every stored row expires after 180 days.
7. **The AI is fenced in.** Whitelisted tools, placeholders instead of personal data, number grounding and a compliance filter, each working without relying on the model to behave.
8. **Advice leaves a tamper-evident trace.** A confirmed plan can be sealed with a salted fingerprint on a public test blockchain.
9. **Failures are safe and visible.** Slow data, an unavailable AI or heavy load lead to a labelled fallback or a plain refusal, never to a wrong number or a leaked detail.

4.2 Suitability and investor protection

- **Know your client.** A validated questionnaire (Grable and Lytton for attitude, Lusardi and Mitchell for knowledge, both quoted verbatim) plus a separate capacity section.
- **Product matches client.** Every product has a risk level from R1 to R5, and a client in category Cn may only hold products up to the level the category allows, within class caps and a stress-loss limit. Complex products need the knowledge check.
- **Independent verification.** The Risk Engine re-checks every holding and fails closed on anything unknown.
- **Records.** The questionnaire answers are recorded with a version and a hash; a profile has a validity date and an expired profile is flagged, never silently reused. The report explains in one sentence how the category was set.
- **Disclosures.** Every simulated number is labelled hypothetical or in-sample; chances are shown as "N in 100 simulated futures"; disclosures follow the style of FINRA Rule 2214; a banner marks the category limits as provisional.
- **Advice, not execution.** There is no order code in the system. The broker link is read-only, and neither the website nor the API ever asks for a broker username or password.

4.3 Identity, sessions and access control

Threat	Control
Weak passwords and brute force	Supabase Auth with hashed passwords, e-mail confirmation, a minimum length of 10 characters, only e-mail sign-in enabled and anonymous sign-ins off. Sign-in and sign-up are rate-limited in the frontend's server routes. Passwords never reach our backend.
Account enumeration through password reset	"Forgot your password?" gives the same answer whether or not an address has an account. The reset link's session is checked with Supabase, used once to set the new password and immediately exchanged for a fresh session; its tokens are removed from the address bar when the page opens.
Session theft	Tokens are kept only in `HttpOnly`, `Secure`, `SameSite=Lax` cookies (with the `__Host-` prefix in production). JavaScript in the page never sees a token.
Cross-site request forgery	Every state-changing account and sign-in route checks the request's `Origin`, and the backend accepts only Bearer tokens, not cookies.
One user reading another's data	No endpoint takes a user id: identity comes only from the verified token. Every database query runs as the user, so Postgres Row-Level Security applies, and also carries the backend's own user filter. There is no service-role key anywhere in the system, and the account store refuses to start if it is given a privileged key.
Direct database access with the public key	Table- and column-level grants (no `UPDATE` or `DELETE` where none is needed; inserts cannot set their own creation or expiry dates) plus "own rows only" policies. On the live project, anonymous requests are refused on every table.
Forged or expired tokens	A token the sign-in service cannot read gets a plain 401, never a server error.

User isolation was tested in three ways: 265 automatic tests in which user A tries every account endpoint against user B's data, including smuggled user ids and forged tokens (`tests/accounts/`); an SQL isolation script run on the live Supabase project, which passed on 19 October (`supabase/tests/rls_isolation.sql`); and probes of the public Data API without a token.

4.4 Data protection and privacy

We keep as little as we can, for a limited time. Every stored row expires after 180 days and a daily job deletes expired rows.

Data	Why it is kept	Removed by
E-mail, password hash, chosen name	Sign-in	Deleting the account
Questionnaire answers, preferences, goal inputs, Risk Game result	Continuing on another device	"Delete my data"
Saved reports	Showing and sealing them	"Delete my data"

Imported trades and cash flows (date, symbol, side, quantity, price and currency only)	Holdings and behaviour	"Delete my data"
Consents, activity log, trade checks, what-if runs	Proof of consent and audit	"Delete my data"
Sealed advice records	Tamper evidence	"Delete my data" (the public fingerprint stays but can no longer be linked to anyone)

- **Never stored:** broker usernames or passwords (never asked for), Interactive Brokers Flex tokens (used for one request, then dropped), uploaded files (parsed in memory), chat text, and API keys in the repository.
- **Consent** is versioned and asked separately before broker data is connected, before trade checks and before sealing. A changed consent text requires a new consent.
- **Rights:** a person can see their data on the Account page, export a report, delete all of their data and delete their account. Erasure is complete by construction: one database function removes every user table, and a test fails if any table with a user id or an expiry date is missing from it (`tests/accounts/test_migrations.py`).
- **Processors:** Vercel and Railway (hosting), Supabase (accounts and data), OpenAI (explanations, redacted, `store=false`), Tiingo and Binance (prices; no personal data is sent) and a blockchain RPC provider (fingerprints only).

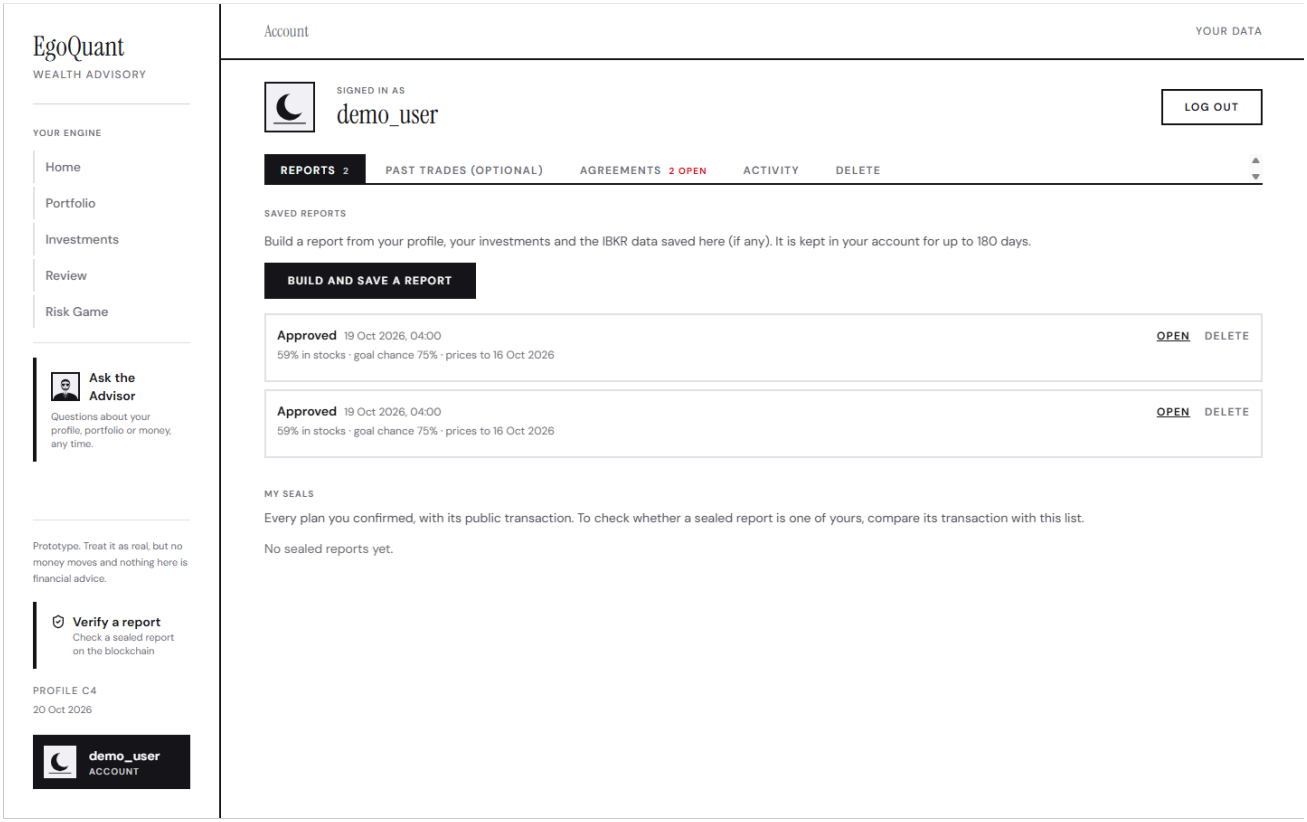


Figure 6. The Account page: saved reports (kept for up to 180 days), with tabs for past trades, agreements (consents), activity and deletion, and the list of the person's own seals.

4.5 AI governance

The advisor is the part of the system most exposed to misuse, so it has the most layers. Each layer works on its own, and none of them depends on the model following its instructions.

Risk	Control
The AI makes a decision	It has no write path to any score, category, limit, weight, approval or verdict. Its only tools run deterministic code with whitelisted arguments, at most two calls per message.
The AI invents a number	Number grounding: every percentage, "N in 100", money amount and multiplier in a reply must come from the engine facts, from this turn's tool results or from the user's own words. Otherwise the reply is regenerated once with a correction and then replaced by a safe fallback (`egoquant/advisor/grounding.py`).
Trade signals or forecasts	A compliance guard checks every sentence, also while the reply is streaming, against 36 patterns in English and Chinese (buy or sell instructions, price targets, guarantees, forecasts) and withholds any match (`egoquant/advisor/guard.py`).

Personal data reaches the model	A redaction gateway replaces account ids, e-mail addresses, phone numbers and the person's name with placeholders before every model call and restores them in the reply; it logs only counts (`egoquant/advisor/redaction.py`). The model never receives balances, holdings lists or raw trades: for a trade question it gets the verdict and its reasons. Calls use `store=false` .
Prompt injection	Chat history and news headlines are treated as untrusted data. The engine facts are HMAC-signed, and a token of another kind (news, what-if) is never accepted as chat facts. Injection-shaped tool arguments such as "use C5 limits" fail the whitelist.
The AI is unavailable	Reports, limits, trade checks and what-if numbers do not depend on it, and explanations fall back to templates.
Silent changes of behaviour	Every call logs the prompt version (currently `advisor-v11`), and a prompt change requires a new live red-team run (`eval/chat_redteam.py`).

The live red-team is a 42-case suite of our own covering what-if questions, market regimes, compliance and 18 trade questions. On 20 October it found a real defect in prompt `advisor-v10` : the model filled both size fields of the trade tool, so the server refused the call and no trade check completed. That failure was safe (no advice was given and nothing leaked). The tool was changed to a single size field the same hour, and `advisor-v11` passed 41 of the 42 cases; the one miss was a false positive of the test's own keyword check on a correct reply. The trade-explanation prompt passed 51 of 51.

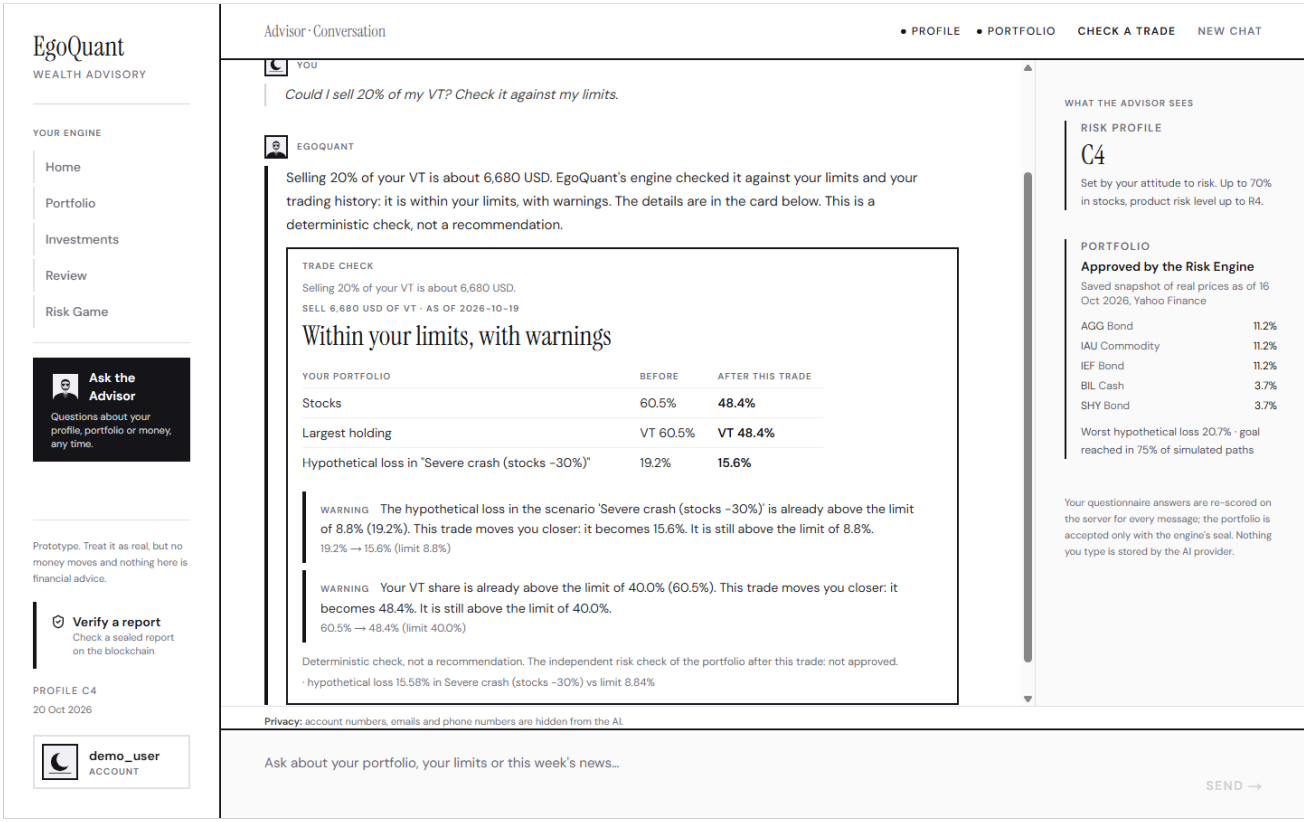


Figure 7. A trade question in the chat. The model only phrases the result; the card underneath is the deterministic engine's check, and the privacy line states what is hidden from the AI.

4.6 Input handling and application hardening

- **Strict schemas.** Every request body is validated against a schema that rejects unknown fields and enforces ranges and lengths.
- **No echo.** A validation error names the field and the rule ("Horizon in years: must be above 0") and never repeats the value that was sent, so a credential pasted into the wrong field cannot come back in a response. Thirteen canary values sent to the live server never came back.
- **Hostile input.** 80 hostile-input cases on the account and trade-check routes (oversized bodies, unicode and null bytes, NaN and Infinity, forged tokens, path-like ids, spreadsheet formulas), 33 more on the Risk Game and 129 extreme but valid investor profiles produce no server error and no NaN in any number.
- **File uploads.** Broker files are parsed with `defusedxml` , with DTDs refused (no external entities, no "billion laughs"); they are limited to 2 MB and 20,000 rows, the extension must match the content, and they are read in memory only. Symbols and currencies must match strict patterns, so formula text cannot get through.
- **Size and rate limits.** Request bodies are capped by size. Portfolio builds and trade checks are limited to 20 a minute, the chat to 120 a minute, account calls to 30 a minute per user, and what-if runs and chat trade checks to 10 per 10 minutes per user. Refusals carry a `Retry-After` header.

- **Headers.** `nosniff`, frame denial, a strict referrer policy, no camera, microphone or location access, HSTS, a content-security policy on the frontend, and CORS limited to the frontend's origin without credentials.
- **Errors.** One generic 500 with no stack trace or exception text; the cause is logged by type and route only. Debug pages answer 404 in production.

4.7 Tamper-evident advice: the on-chain seal

When a signed-in person confirms an approved plan, EgoQuant renders the final report once into a sealed document whose header names the person's chosen username and the report id, and stores it byte for byte. It then builds an advice record in a fixed format: the category, the answers, the portfolio weights to the millionth, the Risk Engine's checks, the code and model versions, the steps the person took and the SHA-256 of the sealed document. A random salt is added, and the fingerprint `SHA-256(salt || record)` is published on the Ethereum Sepolia test network in a transaction that moves no money, labelled `EGOQ1` (`egoquant/anchoring/`, `09_ON_CHAIN_SEAL`).

The person downloads one sealed report. The public Verify page, which needs no account, recomputes both fingerprints in the browser, reads the chain, and answers either "Unchanged", with the block and the date, or "Does not match", meaning the report was changed after it was sealed. A command-line verifier (`python -m egoquant.anchoring verify <file>`) runs the same checks without our website, for anyone who prefers not to trust the page we serve.

- **Privacy.** Only the label and the 32-byte salted fingerprint are public. Without the record and its salt the fingerprint reveals nothing and cannot be tested against guesses. "Delete my data" removes the record, the salt and the sealed document, after which we can no longer link the fingerprint to a person.
- **Safety.** Only the test network is accepted; any other chain id is refused. The sealing key is read only from the server's environment and no public endpoint returns it. Sealing never blocks or changes advice: if the chain is down, the plan is confirmed and the seal waits. Each user may seal 5 reports a day.
- **What it proves.** That this exact report existed, unchanged, no later than that block's time, sealed for that username. It does not prove that the advice was right, and a printed or PDF copy cannot prove itself.
- **Evidence.** Five real seals of real engine output for invented investors, with tampered copies that fail, are in `6_Execution_Evidence/b_onchain`; 465 automatic tests cover the seal format, single-character changes, forged footers and older seals.

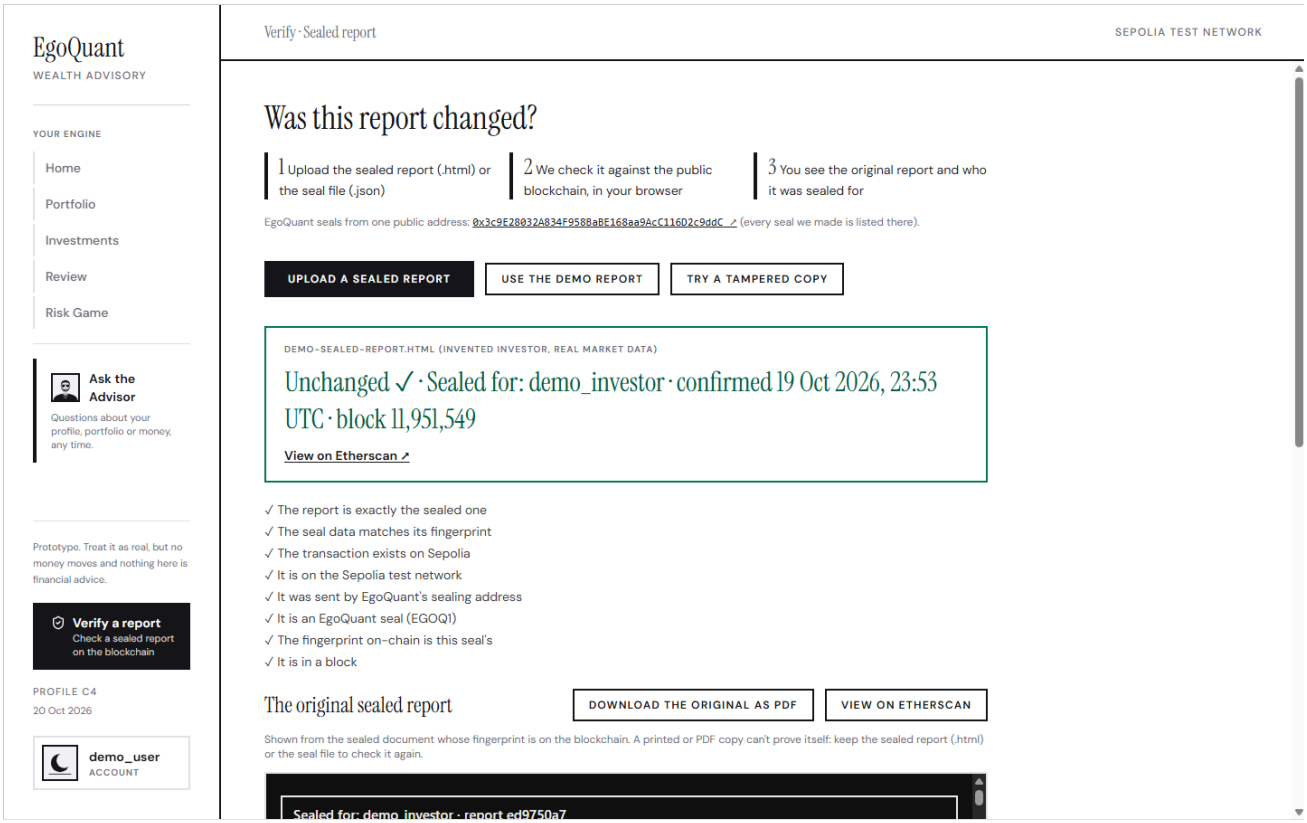


Figure 8. The public Verify page checking a sealed report against the chain, without an account.

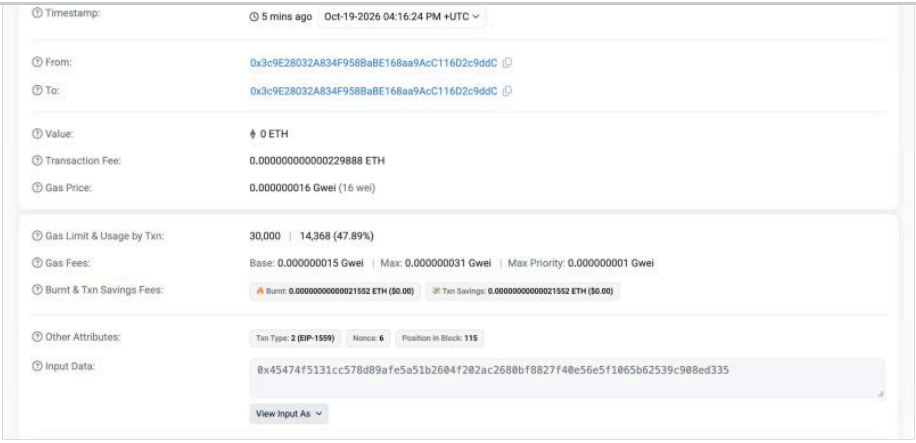


Figure 9. A seal on Sepolia: 0 ETH from the sealing address to itself; the input data is the `EGOQ1` label followed by the 32-byte fingerprint.

In a production deployment the same fingerprint format would go to a consortium chain such as FISCO BCOS.

4.8 Availability and safe failure

- **Price feeds.** A slow feed switches the whole build to the labelled snapshot. The live server runs in snapshot mode and sends no live price request at all.
- **News and AI.** The news briefing has a 5-second budget inside a request; the report is sent without it and the briefing is filled in afterwards.
- **Load.** A build gate allows one heavy build at a time by default and queues the rest, ending in a plain 503 with `Retry-After`. In a local flood test, 20 of the heaviest builds sent at once crashed the server in 3 of 4 runs without the gate; with the gate nothing crashed and memory stayed at about 300 MB.
- **Kill switches.** Every optional part (live prices, the regime sampler, news, the chat, accounts, the Risk Game and the alternative portfolio method) can be switched off with an environment variable, without a code change (`docs/DEPLOY_ENV.md`).
- **Deploy check.** A read-only script (`eval/deploy_check.py`) checks a deployment end to end: status, Python version, price mode, security headers, CORS, the Risk Game on three investors, builds against the demo pack, trade checks and refusals without a token. Its run on 20 October passed 71 of 71 checks.

4.9 Secrets, supply chain and verification

API keys live only in the hosting environment or in a git-ignored `.env` file. Staged changes are scanned for secrets before every push, and there is no privileged database key in the system. Every Python dependency is pinned to a tested version, and a `pip-audit` run on 20 October found no known vulnerability in the 56 pinned packages. Changes reach the deploy branch only after the full automatic test suite has passed, and history is never rewritten with a force-push.

Evidence	Result
Automatic tests	About 3,540 offline backend tests; the counts and the last complete runs are in `04_TESTING_AND_EVIDENCE`.
Isolation on the real database	The SQL isolation script passed on the live Supabase project, and anonymous Data API reads are refused on every table.
Mutation checks	Test authors broke the code on purpose (a wrong blend, a missing cap, a table forgotten in erasure, and others). Every mutation was caught.
Property sweeps	300 random investors through the category rule and 5,000 random trades through the trade check.
Live hostile probe	93 read-only requests against an earlier deployment and 18 more after the latest one: no canary value echoed, security headers on every error status, CORS only for the frontend.
Live AI red-team	`advisor-v11` : 41 of 42 cases. Trade explanation: 51 of 51.
On-chain	Every committed proof verifies and every tampered copy fails (`tests/anchoring/test_committed_proofs.py`).

4.10 Known gaps

- The C1 to C5 limits are placeholders until a bank supplies its own mapping. Every report carries a banner saying so.
- Questionnaire and game answers are self-reported, and the game can be replayed. Its effect is limited to one step and never goes past the capacity cap.
- Confirmation e-mails are sent by Supabase's shared sender, which is slow and rate-limited; judges can use the demo account instead.
- Rate limits are counted per server process, not per IP address.
- The frontend's content-security policy allows inline scripts, which Next.js requires; a nonce-based policy is not in place yet.

- The activity log is a record that a user can add to, not a tamper-proof audit log. The on-chain seal covers the advice record itself.
- Sepolia is a test network and does not promise to keep its entries forever. A production deployment would use a consortium chain.
- Part of the committed price snapshot comes from Yahoo Finance, which is acceptable for the competition only; a commercial product needs a licensed data feed.
- An old market-data API key remains in the private development repository's history and must be rotated.

The complete list with owners is in `06_LIMITATIONS_AND_ROADMAP` and `docs/SECURITY.md`.